

Mô hình neural-guided CVRP: giải thích từng phần & công thức

Tài liệu mô tả phần *mô hình neural* đã viết lại (module hoàn thiện trong khung GA + local search). Mỗi phần gắn với file mã: `cvrp_env.py`, `cvrp_model.py`, `cvrp_data.py`, `train_v2.py`.

1 Mục tiêu tối ưu

Bài toán dùng mục tiêu mở rộng (phạt cả số route lẫn quãng đường):

$$F(\text{nghiệm}) = C \cdot K + \sum_{r \in \mathcal{R}} \ell(r), \quad C = 1000,$$

với K là số route, $\mathcal{R}$ tập route, $\ell(r)$ là độ dài route r (khoảng cách Euclid, làm tròn số nguyên ở khâu chấm điểm).

Lưu ý thang đo. Lúc huấn luyện, tọa độ được chuẩn hóa về $[0, 1]^2$ nên quãng đường $\sim O(1)$. Do đó ta *không* dùng $C = 1000$ khi train mà dùng một hệ số phạt λ trên thang $[0, 1]$ (mục 6).

2 Môi trường (MDP) — `cvrp_env.py`

Một lời giải được sinh tuần tự. Trạng thái tại bước t :

$$s_t = (a_t, \rho_t, \{\delta_i\}_{i=1}^N),$$

trong đó a_t là node hiện tại, $\rho_t \in [0, 1]$ là sức chứa còn lại của xe, $\delta_i \in [0, 1]$ là nhu cầu còn lại của khách i (đã chuẩn hóa: $\delta_i = q_i/Q$, nên một xe đầy có sức chứa = 1). Node 0 là depot.

Mặt nạ khả thi $\mathcal{M}_t$ (`get_mask`).

$$\text{depot cấm} \iff (a_t = 0) \wedge (\text{chưa phục vụ hết}), \quad \text{khách } i \text{ cấm} \iff (\delta_i \leq 0) \vee (\delta_i > \rho_t).$$

Khi đã phục vụ hết tất cả khách: chỉ cho phép về depot.

Chuyển trạng thái (`step`).

$$\text{tới khách } i : \rho \leftarrow \rho - \delta_i, \delta_i \leftarrow 0; \quad \text{về depot} : \rho \leftarrow 1.$$

Phần thưởng.

$$r_t = -d(a_{t-1}, a_t) - \lambda \cdot \mathbb{I}[a_{t-1} = 0 \wedge a_t \neq 0].$$

Mỗi lần đi từ depot ra một khách là *mở một route mới*; đếm số lần đó cho ta K . Tổng phần thưởng một quỹ đạo τ :

$$R(\tau) = \sum_t r_t = - \underbrace{\sum_t d(a_{t-1}, a_t)}_{\text{tổng quãng đường}} - \lambda K.$$

3 Encoder — `cvrp_model.py`

Nhúng đầu vào. Với $M = N + 1$ node:

$$h_0^{(0)} = W_{\text{dep}} x_0, \quad h_i^{(0)} = W_{\text{cus}} [x_i; \delta_i] \quad (i \geq 1),$$

$x_i \in [0, 1]^2$ là tọa độ, δ_i nhu cầu chuẩn hóa. Chiều nhúng $E = 128$.

Lớp self-attention (lặp $L = 3$ lần). Mỗi lớp:

$$\tilde{h} = \text{BN}(h + \text{MHA}(h)), \quad h' = \text{BN}(\tilde{h} + \text{FF}(\tilde{h})),$$

với $\text{FF}(z) = W_2 \text{ReLU}(W_1 z)$ và BN là BatchNorm trên chiều E .

Attention có *distance-bias* kiểu ICAM (điểm mới). Cho mỗi head, điểm tương hợp giữa node i, j :

$$\text{score}_{ij} = \frac{(W_Q^h u_i) \cdot (W_K^h u_j)}{\sqrt{d_k}} + b_{ij}, \quad \boxed{b_{ij} = -\text{softplus}(\alpha_\ell) \log_2(M) d_{ij}}$$

trong đó d_{ij} là khoảng cách Euclid (ma trận tính sẵn trong `reset`), α_ℓ là tham số học được của lớp ℓ (khởi tạo -2 , nên ban đầu $\text{softplus}(\alpha) \approx 0.13$, bias yếu rồi tự mạnh dần). Trọng số attention = $\text{softmax}_j(\text{score}_{ij})$. Ý nghĩa: node càng xa bị giảm chú ý; cường độ tỉ lệ $\log_2(M)$ nên tự thích nghi theo quy mô N — đây là cơ chế giúp generalize cross-scale (train ~ 256 vẫn chạy được tới 400).

4 Decoder & chính sách — `cvrp_model.py`

Vector ngữ cảnh.

$$c_t = [\bar{h}; h_{a_t}; \rho_t] \in \mathbb{R}^{2E+1}, \quad \bar{h} = \frac{1}{M} \sum_i h_i \text{ (graph embedding)}.$$

Glimpse (multi-head). Truy vấn $q = W_Q c_t$, khóa/giá trị từ embeddings; cộng bias từ node hiện tại $b_j = -\text{softplus}(\alpha_d) \log_2(M) d_{a_t, j}$, áp mask, softmax, thu được vector glimpse g_t .

Lớp ra (single-head) + tanh clipping.

$$\hat{u}_j = \frac{(W'_Q g_t) \cdot (W'_K h_j)}{\sqrt{E}} + b_j, \quad u_j = C_{\text{clip}} \tanh(\hat{u}_j), \quad C_{\text{clip}} = 10,$$

rồi gán $u_j = -\infty$ cho node bị cấm. Chính sách:

$$\pi_\theta(a_t = j \mid s_t) = \frac{\exp(u_j)}{\sum_k \exp(u_k)}.$$

Giải mã: *greedy* lấy $\arg \max$, *sampling* bốc theo π_θ .

5 Huấn luyện POMO — `train_v2.py`

Với mỗi instance ta chạy S quỹ đạo khác nhau bằng cách ép node xuất phát $a_0^{(s)}$ là S khách phân biệt. Mỗi quỹ đạo cho return $R^{(s)}$.

Baseline chung (POMO) — không cần mạng baseline.

$$\bar{b} = \frac{1}{S} \sum_{s=1}^S R^{(s)}, \quad A^{(s)} = R^{(s)} - \bar{b} \text{ (advantage).}$$

Hàm mất mát (REINFORCE).

$$\nabla_{\theta} \mathcal{L} = -\frac{1}{BS} \sum_{n=1}^B \sum_{s=1}^S A_n^{(s)} \nabla_{\theta} \log \pi_{\theta}(\tau_n^{(s)}), \quad \log \pi_{\theta}(\tau) = \sum_{t \geq 1} \log \pi_{\theta}(a_t | s_t).$$

Tổng bắt đầu từ $t \geq 1$: **bước ép** $t = 0$ **không đưa vào** (node xuất phát do ta chọn, không phải quyết định của policy).

Tối ưu. Adam; learning rate có warmup rồi cosine theo % thời gian:

$$\eta(t) = \begin{cases} \eta_0 \frac{t+1}{T_{\text{wu}}}, & t < T_{\text{wu}}, \\ \eta_{\min} + (\eta_0 - \eta_{\min}) \frac{1 + \cos(\pi \phi)}{2}, & t \geq T_{\text{wu}}, \end{cases} \quad \phi = \frac{\text{thời gian đã dùng}}{\text{ngân sách}}.$$

Gradient được clip theo chuẩn (≤ 1) và dùng mixed precision (AMP).

6 Giảm số route và quãng đường

Hệ số phạt λ (vehicle_penalty) đưa số route vào reward:

$$\max_{\theta} \mathbb{E}[R] = \max_{\theta} \mathbb{E}[-\text{dist} - \lambda K] \equiv \min_{\theta} \mathbb{E}[\text{dist} + \lambda K],$$

tức policy tối ưu *đúng dạng* F trên thang chuẩn hóa (λ đóng vai C). λ được *ramp* dần để đầu train lo học đi-đường, sau mới ép giảm route:

$$\lambda(\phi) = \lambda_{\max} \min\left(1, \frac{\phi}{\rho_{\text{ramp}}}\right).$$

Vì λ ở thang $[0, 1]$ trong khi $C = 1000$ ở thang gốc, $\lambda_{\max}$ phải được tinh chỉnh (mặc định 0.1). Đánh đổi: λ lớn $\Rightarrow$ ít route hơn nhưng có thể đi vòng hơn.

7 Sinh dữ liệu rộng — cvrp_data.py

Mỗi batch bốc một cấu hình theo 5 trục (bám sơ đồ Set-X làm *sàn* đa dạng):

- **Depot:** {central (0.5, 0.5), eccentric (0, 0), random}.
- **Khách:** {random, cluster (số seed $\sim U\{2, \dots, 8\}$), random-cluster}.
- **Demand** (7 kiểu): unitary; $U[1, 10]$; $U[5, 10]$; $U[1, 100]$; $U[50, 100]$; phụ-thuộc-góc-phần-từ; bimodal (nhiều nhỏ + ít lớn).
- **Route-size** $\leftrightarrow$ **capacity** (mẫu chốt cho số route): với $r \sim U[3, 25]$, nhu cầu thô q_{ij} (i = instance, j = khách), đặt $\bar{q}_i = \frac{1}{N} \sum_j q_{ij}$ (nhu cầu *trung bình* của instance). Khi đó

$$Q_i = \max\left(\text{round}(r \cdot \bar{q}_i), \max_j q_{ij}\right), \quad \delta_{ij} = \frac{q_{ij}}{Q_i}.$$

(max với $\max_j q_{ij}$ đảm bảo $\delta_{ij} \leq 1$: mọi khách vừa một xe rỗng.)

Nhờ couple $r \leftrightarrow Q$, tỉ lệ $\delta = q/Q$ (thứ model thực sự “thấy”) trải đủ các *chế độ mật độ route*, từ ~ 3 tới ~ 25 khách/route — điều kiện cần để model biết giảm K .

8 Curriculum & ngân sách bộ nhớ — `train_v2.py`

Curriculum theo thời gian. Mở rộng dải N dần nhưng luôn giữ size nhỏ:

$$[50, 100] \rightarrow [50, 100, 150, 200] \rightarrow [\dots, 256] \rightarrow [\dots, 320, 400].$$

Ngân sách bộ nhớ. Bộ nhớ đỉnh $\approx B S N^2$ (attention encoder lưu $[BS, H, M, M] \propto N^2$; đồ thị decode $\propto N^2$). Do đó chọn:

$$\text{traj} = \left\lfloor \frac{\text{mem_budget}}{N^2} \right\rfloor, \quad S = \min(N, S_{\max}, \text{traj}), \quad B = \max(B_{\min}, \lfloor \text{traj}/S \rfloor).$$

Nhờ vậy $BSN^2 \approx$ hằng số với mọi N (không OOM ở N lớn). Khi N lớn B tụt về 1, nên dùng *gradient accumulation* để giữ số instance hiệu dụng:

$$g = \max(1, \lfloor \text{target_instances}/B \rfloor) \text{ micro-batch / update.}$$

9 Augmentation $\times 8$ (inference) — `cvrp_env.py`

Khoảng cách Euclid bất biến qua nhóm đối xứng của hình vuông đơn vị, nên với (x, y) ta sinh 8 bản:

$$(x, y), (y, x), (x, 1-y), (y, 1-x), (1-x, y), (1-y, x), (1-x, 1-y), (1-y, 1-x).$$

Chạy model trên cả 8 bản, lấy nghiệm tốt nhất mỗi instance $\Rightarrow$ chất lượng cao hơn mà không cần train thêm.

10 Tích hợp trong GA (inference)

Trong crossover, sau khi thừa kế các route tốt còn tập khách chưa gán U . Ta gọi `model.solve(env)` trên subproblem gồm depot + U (tọa độ chuẩn hóa về bounding box), nhận chuỗi action. Mỗi lần gắp depot (0) là mở route mới:

$$\text{actions} = [c_1, c_2, 0, c_3, c_4, c_5, 0, \dots] \Rightarrow \{[c_1, c_2], [c_3, c_4, c_5], \dots\}.$$

Các route này được nối vào nghiệm con; local search (`relocation`, `route_elimination`, `k_opt`, ...) tinh chỉnh tiếp.

Ký hiệu: N số khách, $M = N + 1$, E chiều nhúng, H số head, L số lớp encoder, K số route, S số start POMO, B số instance/micro-batch, λ phạt route, ϕ tỉ lệ thời gian đã dùng, d_{ij} khoảng cách Euclid.